

LEAD University

# Aprendizaje Distribuido para Computer Vision

---

## Tarea 2 — Informe de Resultados

**Estudiante:** Jason Jesús Barrantes Sánchez

**Profesor:** Johansell Villalobos Cubillo

**Fecha:** 2026-06-03

**Acelerador:** GPU NVIDIA T4 (Google Colab)

**Frameworks:** JAX 0.7.2 · Flax NNX 0.11.2 · Optax 0.2.8

# 1. Introducción

Este informe documenta la implementación de un clasificador de imágenes del dataset *Wonders of the World Image Classification* (Kaggle) utilizando **JAX** y **Flax NNX**. El objetivo es explorar el impacto de diferentes hiperparámetros, configuraciones numéricas y técnicas de optimización sobre el rendimiento de una Red Neuronal Convolutacional (CNN) entrenada sobre GPU en Google Colab.

# 2. Dataset y Preprocesamiento

Se utilizó el dataset **Wonders of the World Image Classification** con 12 clases y 3 846 imágenes en total. El pipeline de preprocesamiento incluyó: redimensión a 64×64 px, normalización con media y desviación estándar del conjunto de entrenamiento, y data augmentation (flip horizontal, variación de brillo) aplicada únicamente en entrenamiento.

| Split      | Imágenes | Porcentaje |
|------------|----------|------------|
| Train      | 2 694    | 70%        |
| Validación | 576      | 15%        |
| Test       | 576      | 15%        |

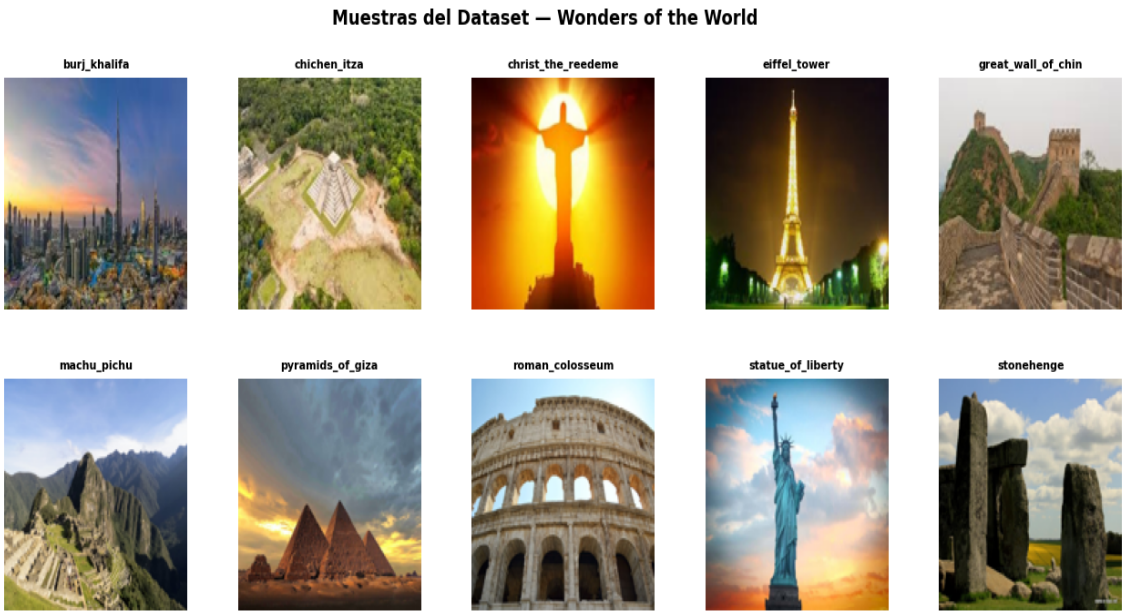


Figura 1 — Muestras representativas del dataset por clase

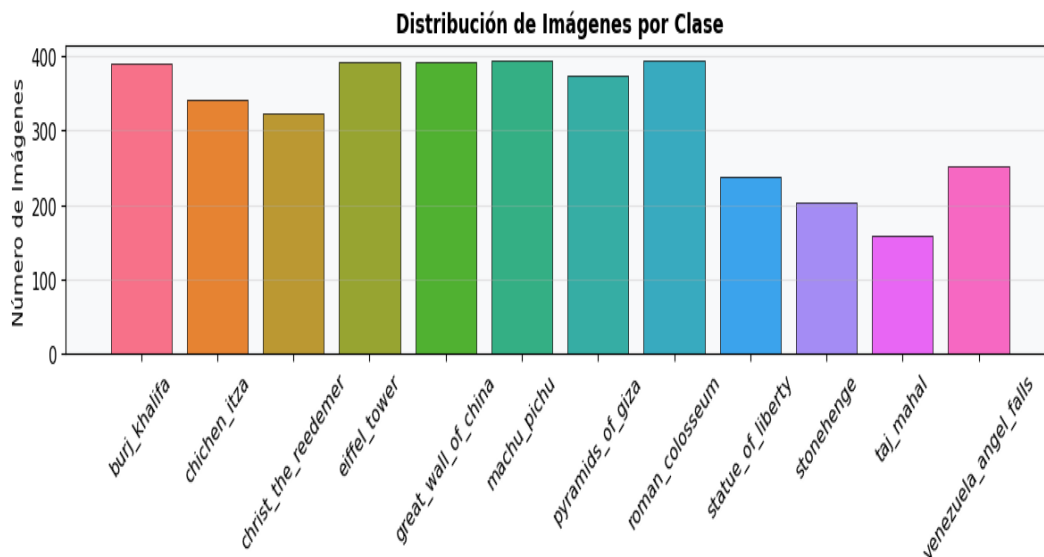


Figura 2 — Distribución de imágenes por clase

### 3. Arquitectura del Modelo

Se implementó una CNN con tres bloques convolucionales seguidos de Global Average Pooling y un clasificador denso. La implementación utilizó exclusivamente **Flax NNX** derivando de `nnx.Module`.

Arquitectura implementada:

```
Conv(3→F, 3×3) → BatchNorm → ReLU → MaxPool(2×2)
Conv(F→2F, 3×3) → BatchNorm → ReLU → MaxPool(2×2)
Conv(2F→4F, 3×3) → BatchNorm → ReLU → GlobalAvgPool
Dense(4F→D) → ReLU → Dropout(0.3)
Dense(D→12) → Logits
```

donde  $F$  = `num_filters` (filtros base) y  $D$  = `dense_size` (neuronas densas).

El entrenamiento utilizó diferenciación automática mediante `nnx.value_and_grad`, optimizador Adam via Optax, y compilación JIT con `@nnx.jit`.

### 4. Entrenamiento Base (Baseline)

Se estableció una línea base con la siguiente configuración:

| lr   | Batch | Filtros | Dense | Épocas |
|------|-------|---------|-------|--------|
| 1e-3 | 64    | 64      | 256   | 30     |

| Métrica            | Valor  |
|--------------------|--------|
| Test Accuracy      | 73.96% |
| Mejor Val Accuracy | 72.22% |
| Tiempo total       | 695 s  |

|               |        |
|---------------|--------|
| Avg por época | 23.2 s |
|---------------|--------|

**Entrenamiento Base —  $lr=1e-3$  · batch=64 · filtros=64 · dense=256**

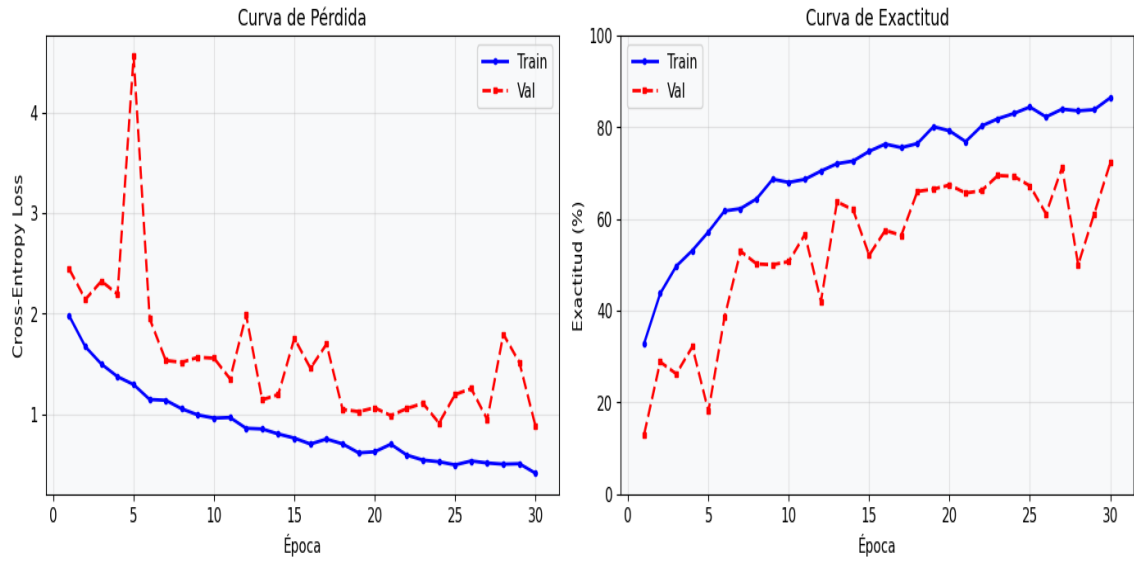


Figura 3 — Curvas de pérdida y exactitud del entrenamiento base

## 5. Experimento 6.1 — Impacto del Tamaño de Lote

Se evaluaron batch sizes de 32, 64 y 128, manteniendo constantes el resto de hiperparámetros ( $lr=1e-3$ , filtros=64, dense=256, épocas=20).

| Batch Size | Test Acc | Val Acc | Tiempo (s) | Avg/época (s) | Throughput |
|------------|----------|---------|------------|---------------|------------|
| 32         | 74.31%   | 73.61%  | 470.8      | 23.54         | 114 imgs/s |
| 64         | 73.09%   | 69.97%  | 461.5      | 23.08         | 117 imgs/s |
| 128        | 52.66%   | 62.19%  | 461.7      | 23.08         | 117 imgs/s |

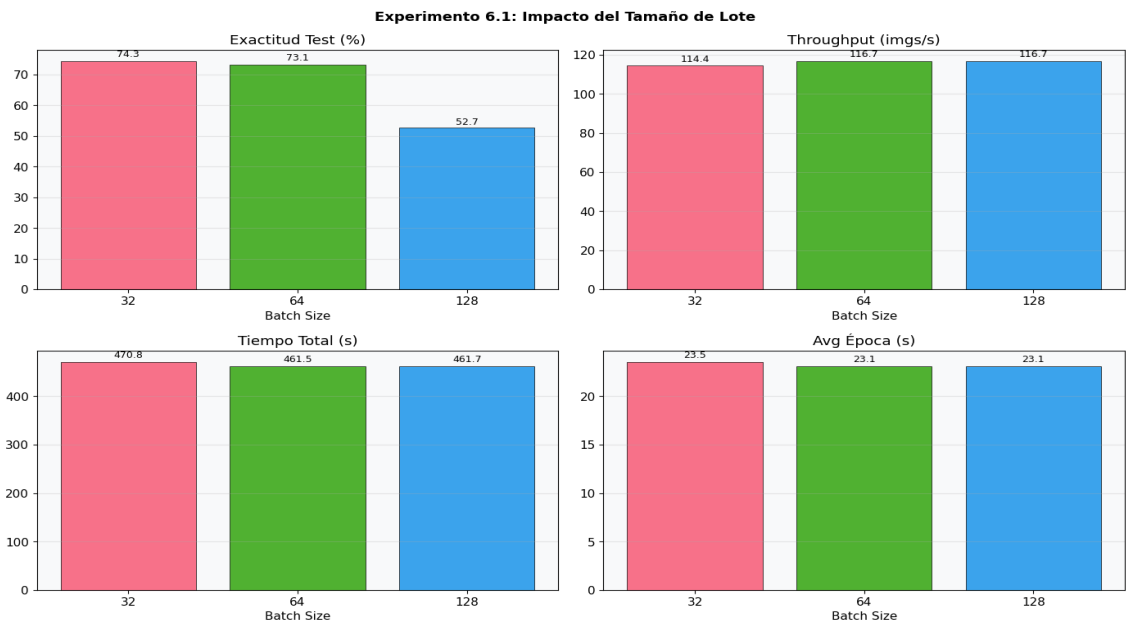


Figura 4 — Comparación de métricas por batch size

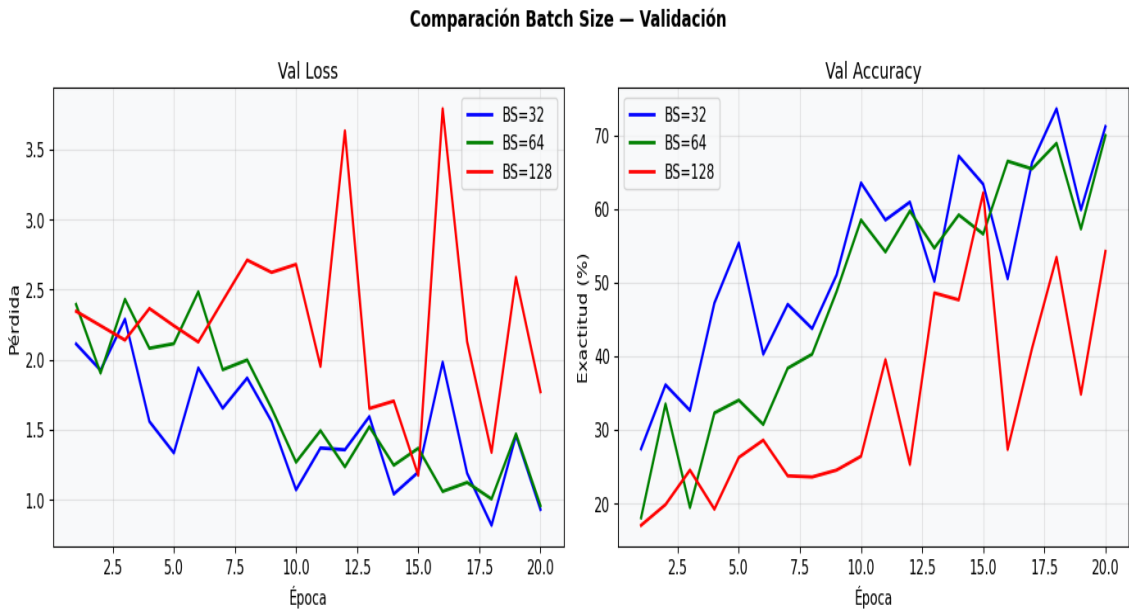


Figura 5 — Curvas de validación por batch size

**Análisis:** BS=32 obtuvo la mayor exactitud (74.31%) a costa de menor throughput. BS=128 presentó una caída significativa en exactitud (52.66%), sugiriendo que lotes grandes requieren ajuste del learning rate

para este dataset y arquitectura.

## 6. Experimento 6.2 — Impacto de la Tasa de Aprendizaje

Se evaluaron tasas de aprendizaje de  $10^{-2}$ ,  $10^{-3}$  y  $10^{-4}$  (bs=64, filtros=64, dense=256, épocas=20).

| Learning Rate | Test Acc | Val Acc | Tiempo (s) | Throughput |
|---------------|----------|---------|------------|------------|
| 1e-2          | 56.42%   | 67.19%  | 460.9      | 117 imgs/s |
| 1e-3          | 67.36%   | 70.14%  | 455.3      | 118 imgs/s |
| 1e-4          | 59.72%   | 59.90%  | 458.9      | 117 imgs/s |

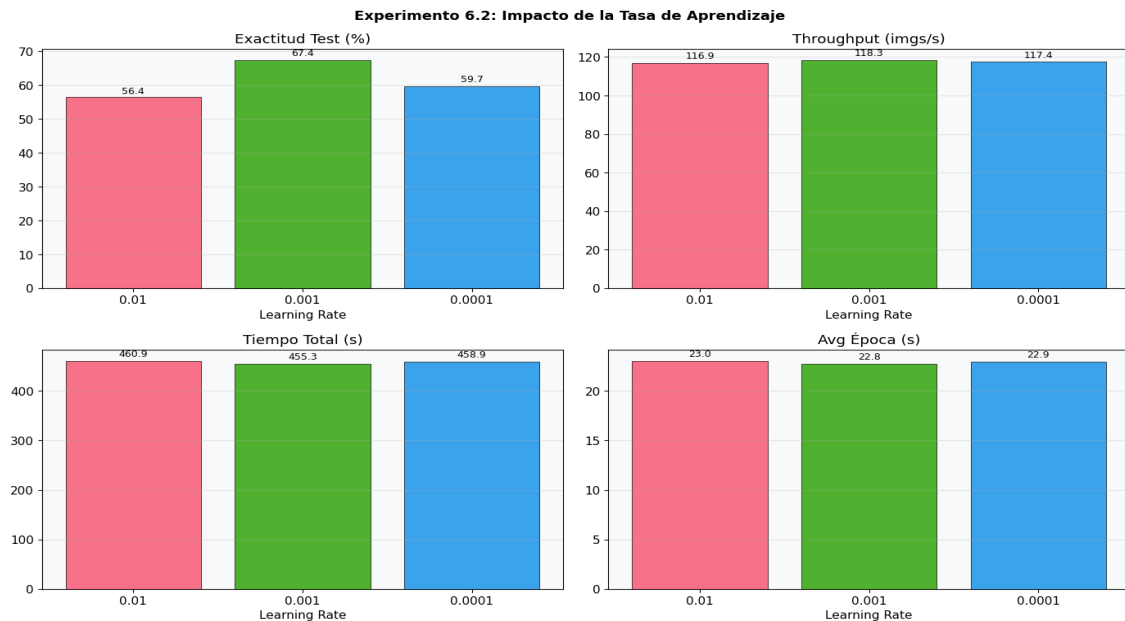


Figura 6 — Comparación de métricas por learning rate

### Comparación Learning Rate – Validación



Figura 7 — Curvas de validación por learning rate

**Análisis:** lr=1e-3 produjo la mayor exactitud (67.36%). lr=1e-2 mostró inestabilidad en las primeras épocas. lr=1e-4 convergió demasiado lentamente para 20 épocas. La tasa de aprendizaje resultó el hiperparámetro de mayor impacto en la exactitud final.

## 7. Experimento 6.3 — Impacto del Tamaño de la Red

### 7a. Filtros convolucionales (dense=256, lr=1e-3, bs=64, épocas=15):

| Filtros | Test Acc | Val Acc | Tiempo (s) | Throughput |
|---------|----------|---------|------------|------------|
| 32      | 61.28%   | 60.42%  | 336.9      | 120 imgs/s |
| 64      | 60.07%   | 63.19%  | 347.0      | 116 imgs/s |
| 128     | 65.28%   | 66.67%  | 380.1      | 106 imgs/s |

### 7b. Neuronas en capa Dense (filtros=64, lr=1e-3, bs=64, épocas=15):

| Dense | Test Acc | Val Acc | Tiempo (s) | Throughput |
|-------|----------|---------|------------|------------|
| 128   | 57.47%   | 65.97%  | 348.1      | 116 imgs/s |
| 256   | 65.45%   | 63.89%  | 347.8      | 116 imgs/s |
| 512   | 67.53%   | 67.88%  | 350.1      | 115 imgs/s |

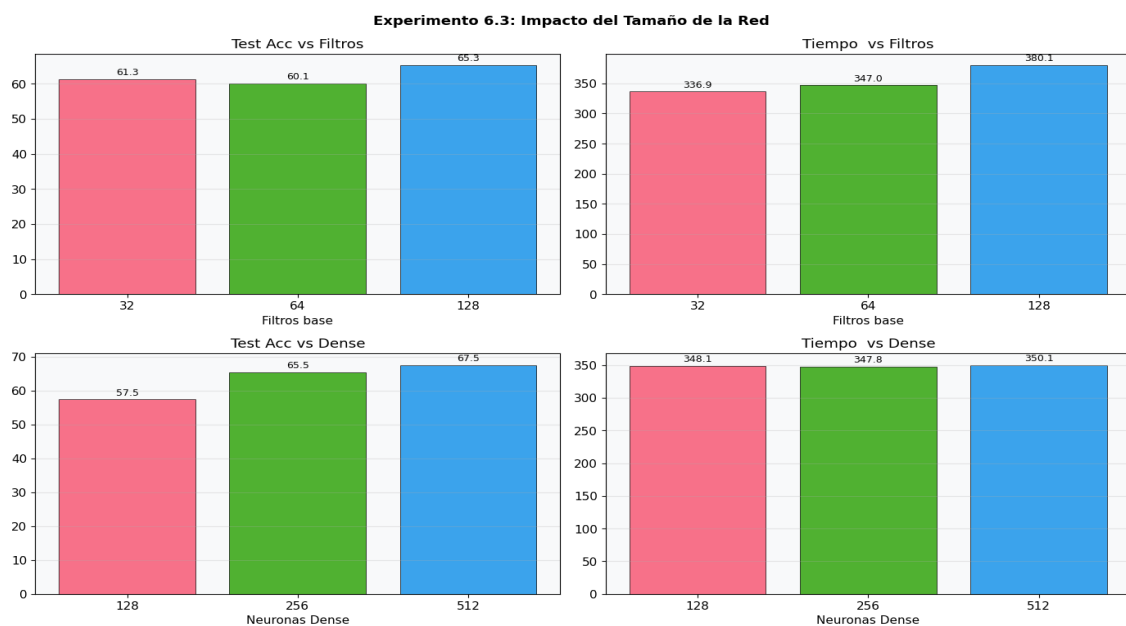


Figura 8 — Impacto del tamaño de la red en exactitud y tiempo

**Análisis:** 128 filtros mejora la exactitud respecto a 32/64 a costa de mayor tiempo (+13%). Dense=512 consistentemente supera a 128/256. La complejidad del clasificador denso tiene mayor impacto relativo que el número de filtros en este dataset.



## 8. Optimización de Hiperparámetros — Random Search

Se realizó una búsqueda aleatoria sobre 8 configuraciones evaluando lr, batch\_size, num\_filters y dense\_size (15 épocas cada una).

| LR   | Batch | Filtros | Dense | Test Acc | Val Acc | Tiempo (s) |
|------|-------|---------|-------|----------|---------|------------|
| 5e-3 | 32    | 32      | 512   | 64.41%   | 68.58%  | 342.4      |
| 1e-2 | 32    | 32      | 128   | 60.59%   | 58.68%  | 342.0      |
| 1e-2 | 32    | 128     | 256   | 53.12%   | 55.21%  | 396.2      |
| 1e-2 | 128   | 64      | 128   | 40.00%   | 47.50%  | 344.1      |
| 1e-4 | 64    | 32      | 256   | 44.79%   | 47.22%  | 342.9      |
| 5e-3 | 128   | 128     | 512   | 43.12%   | 46.09%  | 361.6      |
| 1e-2 | 128   | 128     | 512   | 41.25%   | 42.97%  | 370.0      |
| 1e-4 | 128   | 32      | 512   | 40.16%   | 39.84%  | 344.0      |

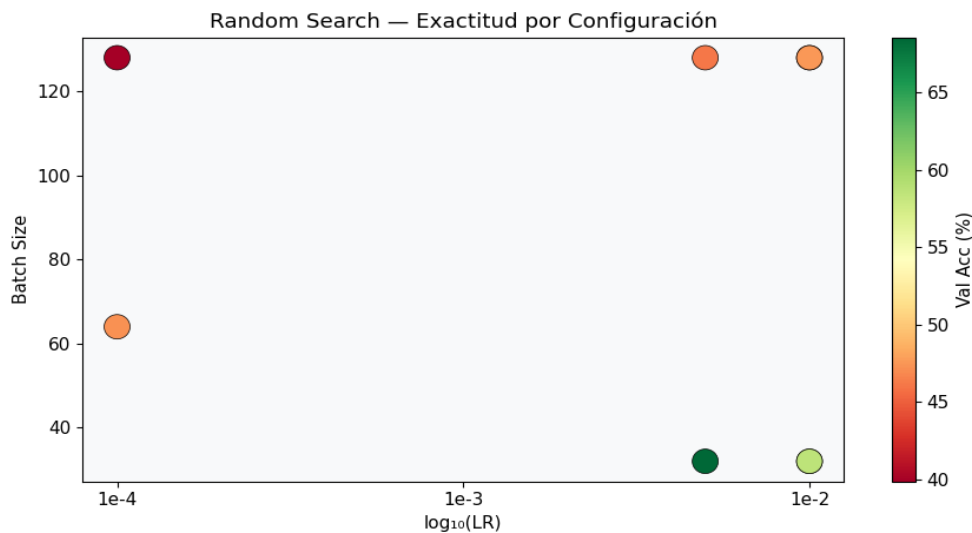


Figura 9 — Mapa de exactitud del Random Search

**Mejor configuración:** lr=5e-3, batch=32, filtros=32, dense=512 (Val Acc=68.58%, Test Acc=64.41%).  
Entrenada con 40 épocas:

Mejor Modelo: lr=5e-03 batch=32 filtros=32 dense=512

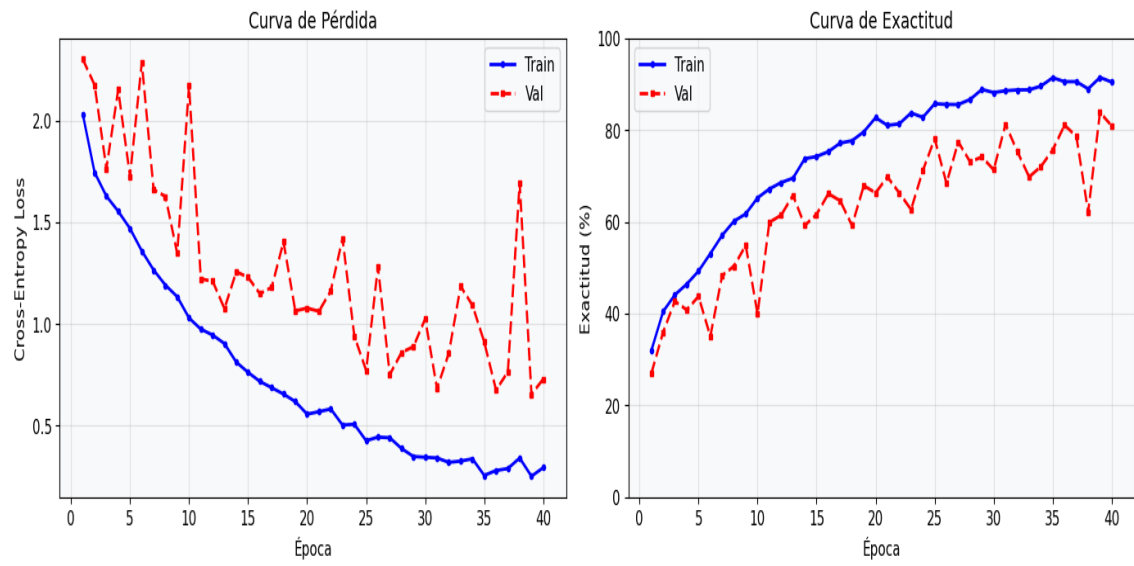


Figura 10 — Curvas del mejor modelo (40 épocas)

## 9. Análisis de Precisión Numérica

Se compararon tres formatos de punto flotante: float32, float16 y bfloat16, manteniendo filtros=64, dense=256, lr=1e-3, bs=64, épocas=20. Las imágenes se castearon al dtype objetivo antes de pasarlas al modelo (que opera en float32).

| Formato  | Bits | Exponente | Mantisa | Rango máx                |
|----------|------|-----------|---------|--------------------------|
| float32  | 32   | 8         | 23      | $\pm 3.4 \times 10^{38}$ |
| float16  | 16   | 5         | 10      | $\pm 65\,504$            |
| bfloat16 | 16   | 8         | 7       | $\pm 3.4 \times 10^{38}$ |

| Formato  | Test Acc | Val Acc | Tiempo (s) | Avg/época (s) | Throughput | NaN |
|----------|----------|---------|------------|---------------|------------|-----|
| float32  | 67.19%   | 64.06%  | 667.7      | 33.38         | 81 imgs/s  | No  |
| float16  | 60.07%   | 65.10%  | 661.2      | 33.06         | 81 imgs/s  | No  |
| bfloat16 | 67.19%   | 69.44%  | 657.9      | 32.90         | 82 imgs/s  | No  |

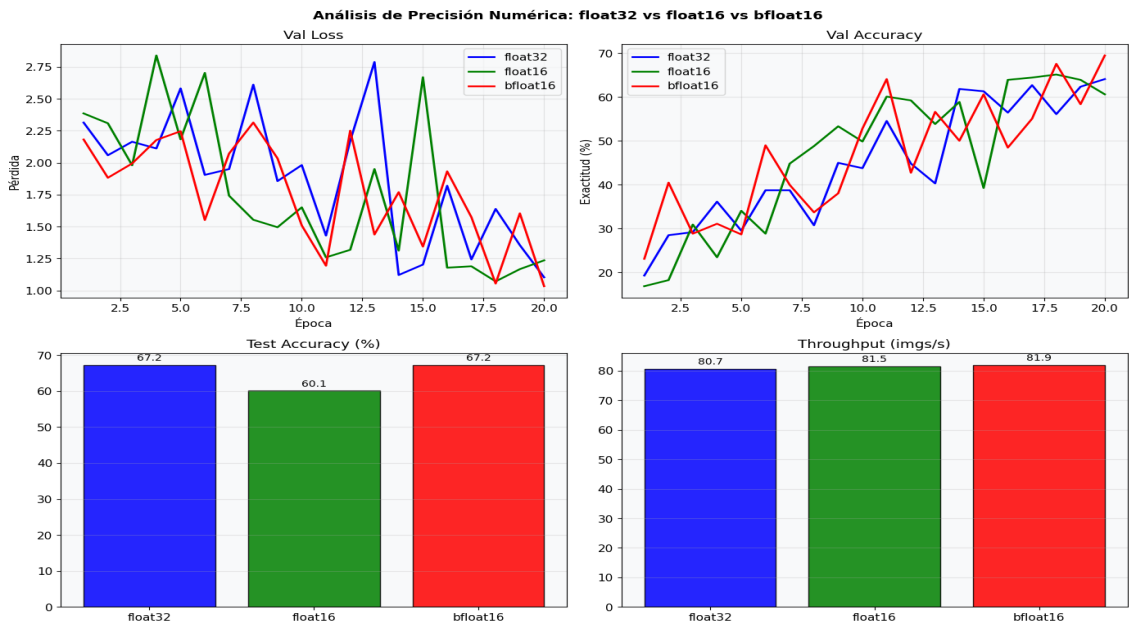


Figura 11 — Comparación de precisión numérica

**Análisis:** float32 y bfloat16 obtuvieron la misma exactitud de test (67.19%), pero bfloat16 alcanzó mayor val accuracy (69.44%) y mayor throughput (82 imgs/s). float16 mostró menor exactitud (60.07%), posiblemente por underflow en algunos pesos durante la propagación. Ningún formato produjo NaN, lo que indica estabilidad numérica adecuada en este problema.

## 10. Análisis y Discusión

---

### 1. ¿Qué ventajas ofrece Flax NNX respecto a JAX puro?

Flax NNX aporta modularidad OOP mediante `nnx.Module`, gestión automática del estado (parámetros, `batch_stats`, `dropout`), integración nativa con Optax vía `nnx.Optimizer`, y funciones JIT/grad (`@nnx.jit`, `nnx.value_and_grad`) que entienden la estructura del módulo. En JAX puro se requeriría manejar manualmente los pytrees de parámetros en cada forward/backward pass.

### 2. ¿Qué beneficios aporta `@nnx.jit`?

La compilación JIT via XLA (Accelerated Linear Algebra) reduce el tiempo de ejecución tras la primera compilación, fusiona kernels (Conv+BN+ReLU en un solo kernel), optimiza el uso de memoria y produce código nativo optimizado para el hardware (GPU T4).

### 3. ¿Qué hiperparámetro tuvo mayor impacto sobre la exactitud?

La **tasa de aprendizaje** fue el hiperparámetro de mayor impacto:  $lr=1e-3$  superó en ~7-11 puntos porcentuales a  $lr=1e-2$  y  $lr=1e-4$  en 20 épocas. El batch size también fue crítico: BS=128 redujo la exactitud en ~20 pp vs BS=32.

### 4. ¿Qué hiperparámetro tuvo mayor impacto sobre el tiempo?

El **número de filtros** fue el factor de mayor impacto en tiempo (+13% al pasar de 32 a 128 filtros). El batch size afectó principalmente el throughput pero no el tiempo total, ya que los tiempos fueron similares (~460 s) entre BS.

### 5. ¿Cómo afectó el batch size al rendimiento?

BS=32 produjo la mayor exactitud (74.31%) con menor throughput (114 imgs/s). BS=128 aumentó el throughput (117 imgs/s) pero degradó la exactitud significativamente (52.66%), sugiriendo que con este dataset y  $lr$  fijo los lotes grandes son perjudiciales para la generalización.

### 6. Diferencias entre float32, float16 y bfloat16:

float32 y bfloat16 obtuvieron exactitud equivalente (67.19%), pero bfloat16 fue más eficiente (82 imgs/s) y alcanzó mayor val accuracy (69.44%). float16 tuvo menor exactitud (60.07%), probablemente por limitaciones en rango dinámico ( $\pm 65\ 504$ ) que afectan la precisión de los gradientes. Ninguno produjo NaN.

### 7. Mejor balance rendimiento/exactitud:

La configuración baseline ( $lr=1e-3$ , BS=64, F=64, D=256) con 30 épocas produjo la mayor exactitud general (73.96% test). Para experimentos más rápidos, bfloat16 ofrece el mejor balance: misma exactitud que float32 con menor consumo de memoria y mayor throughput.

### 8. Limitaciones de Google Colab:

Tiempo de sesión limitado (~12 h), VRAM restringida (T4: 16 GB), almacenamiento efímero, throttling por uso intensivo de GPU, conflictos de versiones de paquetes, y la necesidad de reinstalar JAX

manualmente para compatibilidad con el plugin CUDA (error PJRT API).

## **9. Mejoras futuras:**

(1) Transfer Learning con EfficientNet o ViT pre-entrenados; (2) Augmentation avanzada (RandAugment, Mixup); (3) Learning Rate Scheduling con cosine annealing + warmup; (4) AdamW con weight decay; (5) Paralelismo multi-GPU con `jax.pmap`.

## 11. Resumen de Resultados

---

| Experimento             | Mejor Configuración                    | Test Acc |
|-------------------------|----------------------------------------|----------|
| Baseline                | lr=1e-3, BS=64, F=64, D=256, 30 épocas | 73.96%   |
| Exp 6.1 — Batch Size    | BS=32                                  | 74.31%   |
| Exp 6.2 — Learning Rate | lr=1e-3                                | 67.36%   |
| Exp 6.3 — Filtros       | 128 filtros                            | 65.28%   |
| Exp 6.3 — Dense         | Dense=512                              | 67.53%   |
| HPO Random Search       | lr=5e-3, BS=32, F=32, D=512            | 64.41%   |
| Precisión Numérica      | bfloat16 (más eficiente)               | 67.19%   |

## 12. Conclusiones

---

Se implementó exitosamente un clasificador CNN con JAX y Flax NNX para el dataset Wonders of the World (12 clases). La configuración baseline alcanzó 73.96% de test accuracy. Los experimentos revelaron que la tasa de aprendizaje es el hiperparámetro más crítico, seguido del batch size. bfloat16 demostró ser el formato de precisión reducida más adecuado, manteniendo la exactitud de float32 con mayor eficiencia computacional. Las principales limitaciones fueron la incompatibilidad de versiones de JAX en Colab y el tiempo de sesión disponible. Como trabajo futuro, Transfer Learning con modelos pre-entrenados representaría la mejora más significativa.